



slington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CC5009NI Cyber Security in Computing

Assessment Weightage & Type

60% Group Coursework 02

Year and Semester

2025 -26 Autumn Semester

Student Name	London Met ID
Abinash Kumar Sah	23056140
Suraj Tiwari	24046524
Gaurab Upadhyaya	24046385
Nisha Yadav	24046546

Assignment Due Date: Monday, Apr 20, 2026

Assignment Submission Date: Friday, May 15, 2026

Word Count (Where Required):6900

I confirm that I understand my coursework needs to be submitted online via MST under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Document Details

Submission ID
trn:oid::3618:138979374

Submission Date
May 14, 2026, 8:36 PM GMT+5:45

Download Date
May 14, 2026, 8:37 PM GMT+5:45

File Name
23056140 Abinash Kumar Sah.pdf

File Size
2.2 MB

39 Pages

6,933 Words

41,467 Characters



Page 1 of 44 - Cover Page

Submission ID trn:oid::3618:138979374



Page 2 of 44 - Integrity Overview

Submission ID trn:oid::3618:138979374

10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Small Matches (less than 8 words)

Match Groups

-  **42 Not Cited or Quoted 9%**
Matches with neither in-text citation nor quotation marks
-  **11 Missing Quotations 2%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

-  **3% Internet sources**
-  **1% Publications**
-  **10% Submitted works (Student Papers)**

Match Groups

- 42 Not Cited or Quoted 9%**
Matches with neither in-text citation nor quotation marks
- 11 Missing Quotations 2%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%** Internet sources
- 1%** Publications
- 10%** Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works		
	Islington College, Nepal on 2026-01-18		3%
2	Submitted works		
	Asia Pacific University College of Technology and Innovation (UCTI) on 2025-04-03		<1%
3	Submitted works		
	Asia Pacific University College of Technology and Innovation (UCTI) on 2025-11-02		<1%
4	Submitted works		
	Southern New Hampshire University - Continuing Education on 2025-04-22		<1%
5	Submitted works		
	Crown Institute of Business and Technology on 2025-09-14		<1%
6	Submitted works		
	RDI Distance Learning on 2025-02-18		<1%
7	Internet		
	0xc0ffee.netlify.app		<1%
8	Submitted works		
	Asia Pacific University College of Technology and Innovation (UCTI) on 2024-05-31		<1%
9	Internet		
	zestlab.io		<1%
10	Submitted works		
	University of Westminster on 2025-12-19		<1%

Table of Contents

1. Introduction	8
1.1 Introduction to the topic	1
1.2 Overview of Injection Flaws	2
1.3 Significance and Impact of SQL Injection	3
1.4 Technical Terminologies	4
1.5 Aim and Objectives	5
1.6 Report Structure	6
2. Background	7
2.1 Overview of Injection Flaws Vulnerabilities.....	7
2.2 Overview of SQL Injection	8
2.3 Types of SQL Injection Attacks	8
2.3.1 In-Band SQL Injection	8
2.3.2 Inferential (Blind) SQL Injection.....	9
2.3.3 Out-of-Band SQL Injection	9
2.4 Causes of SQL Injection Vulnerabilities	10
2.5 Tools and Techniques for Identifying SQL Injection.....	10
2.6 Consequences of SQL Injection.....	11
2.7 Real-World Incidents of SQL Injection.....	11
2.8 Penetration Testing Execution Standard (PTES)	11
2.9 Evolving Attack Methods	12
2.10 Modern Defence Strategies	12
3. Demonstration of Injection Flaws in Web Applications Using OWASP Juice Shop	13
3.1 Introduction to the Practical Demonstration	13
3.2 Practical Laboratory Environment Setup	13
3.3 Reconnaissance and Target Identification	15
3.4 Baseline Authentication Testing	16
3.5 Payload Preparation and Technical Analysis	17
3.6 Exploitation and Authentication Bypass	18
3.7 HTTP Request and API Layer Analysis	19
3.8 Token Analysis and Session Validation	20
3.9 Validation of Authenticated Requests.....	21
3.10 Manual API Testing using Postman	22
3.11 Automated Testing using sqlmap.....	22

3.12 Defensive Assessment using OWASP ZAP	23
3.13 Critical Evaluation of the Demonstration.....	24
4. Mitigation of Injection Flaws in Web Applications	25
4.1 Prepared Statements / Parameterized Queries	25
4.2 Input Validation and Sanitization	25
4.3 Web Application Firewall (WAF) – Practical Implementation	25
4.3.1 Normal Request Behaviour	26
4.3.2 Attack Request Behaviour (Blocked)	27
4.3.3 Comparison: Before vs After Mitigation	28
4.4 Least Privilege for Database Accounts.....	29
4.5 Proper Error Handling.....	29
4.6 Secure API Design	30
4.7 Regular Security Testing and Monitoring	30
4.8 Strengths and Limitations of Mitigation Techniques	31
5. Evaluation	32
5.1 Prepared Statement / Parameterized Queries	32
5.2 Input Validation and Sanitization	33
5.3 Web Application Firewall (WAF)	34
5.4 Least Privilege for Database Accounts.....	35
5.5 Proper Error Handling.....	36
6. Conclusion.....	38
References	39

Table of Figure

Figure 1: SQL Injection Attack	1
Figure 2: Overview of Injection Flaws	2
Figure 3: CIA Triad	3
Figure 4: SQL Injection Mechanism	7
Figure 5: OWASP Juice Shop Running on Kali Linux	14
Figure 6: OWASP Juice Shop Running at Host	14
Figure 7: Identification of the /rest/user/login API endpoint using browser Developer Tools	16
Figure 8: Baseline authentication test showing invalid credentials response	17
Figure 9: SQL Injection Payload Prepared for Authentication Bypass Testing	18
Figure 10: Successful authentication bypass using injection-style payload	18
Figure 11: Captured HTTP POST Login Request Containing Injection Payload.....	19
Figure 12: JWT Token Returned after Successful Authentication Bypass	20
Figure 13: Decoded JWT Token Showing Administrator Session Claims	20
Figure 14: Authenticated API Request using JWT Bearer Token	21
Figure 15: Manual API Testing in Postman Demonstrating	21
Figure 16: Manual API Testing in Postman Demonstrating Successful Authentication Bypass	22
Figure 17: Automated SQL Injection Testing using sqlmap	23
Figure 18: OWASP ZAP Vulnerability Scanning Results	24
Figure 19: Python Flask Web Application Firewall Running on Port 8080	26
Figure 20: Legitimate Login Request Allowed through WAF Returning 401 Unauthorized	27
Figure 21: Injection Payload Blocked by WAF Returning 403 Forbidden	28
Figure 22: Successful Authentication Bypass before WAF Implementation	28
Figure 23: Injection Payload Blocked after WAF Implementation.....	29

Table of Tables

Table 1: Technical Terminologies and Definitions	4
Table 2: Types and Detection Methods of SQL Injection Attacks.....	9
Table 3: Tools and Their Purpose in the SQL Injection Testing Environment	15
Table 4: Observed HTTP Request Properties for the Login Endpoint.....	19
Table 5: Strengths and Limitations of SQL Injection Mitigation Techniques	31

Abstract

This coursework explores in detail injection vulnerabilities in modern Web applications, emphasizing SQL Injection vulnerabilities and injection-based authentication bypass attacks. Injection vulnerabilities continue to represent one of the most critical security threats affecting modern web applications since the attacker can easily control backend systems with the use of malicious content from the users. Injection attacks are thoroughly discussed and analyzed in the report, which covers the causes, types, effects, and prevention methods of injection attacks, supported by theoretical analysis and practical security testing.

A practical demonstration was carried out in an ethical laboratory environment using OWASP Juice Shop. Authenticating mechanisms, testing malicious payloads, and assessing application security were analyzed using multiple security testing tools such as Postman, SQLmap, OWASP ZAP and Browser Developer Tools. The demonstration illustrated the potential for unauthorized access and JWT token creation due to insecure input handling and poor API validation. The report also pointed out the drawbacks of using only automated vulnerability scanning tools.

Various mitigation mechanisms were explored, such as parameterized queries, Web Application Firewalls (WAFs), least privilege, proper error handling, and secure API design and architecture. A practical WAF implementation was also created and tested and successfully rejected malicious traffic while allowing legitimate traffic.

Overall, the class work illustrates that injection vulnerabilities remain significant threats to today's web applications. To ensure the applications are well protected, it is important to use secure coding, multilayered defense measures, monitoring and regular security testing to minimize the danger of exploitation and enhance the security of the application.

1. Introduction

1.1 Introduction to the topic

Web applications are widely used to handle sensitive data and online transactions, making them a major target for cyberattacks. Injection flaws are critical security vulnerabilities where untrusted input is processed as part of a command or query, leading to unauthorized access and data breaches. Consequently, attackers can alter application behavior, obtain unauthorized access, steal data, and destroy systems (OWASP, 2021).

Various technologies such as databases, operating systems, directory services, and XML processors may be vulnerable to injection attacks. The most common types include SQL Injection, Command Injection, LDAP Injection, NoSQL Injection, and XML/Xpath Injection. One of the most important of these is SQL Injection (SQLi) as most web applications use relational databases to store valuable information. SQL Injection occurs when attackers inject dangerous SQL statements into input boxes like login forms, search boxes, URLs, or API requests. Attackers can circumvent authentication, steal confidential records, alter data stored, or destabilize services (Halfond, 2006).



Figure 1: SQL Injection Attack

1.2 Overview of Injection Flaws

Injection vulnerabilities occur when an application sends untrusted input to an interpreter such as a database, shell, or XML parser without proper validation. The interpreter may execute the malicious input as a valid instruction leading to unwanted behavior like data leakage, unauthorized alteration of records, authentication bypass, or a total compromise of the system (OWASP, 2021).

The OWASP Top 10 (2021) categorized injection flaws in the A03:2021 Injection category as this remains prevalent and critical across organizations regardless of size.

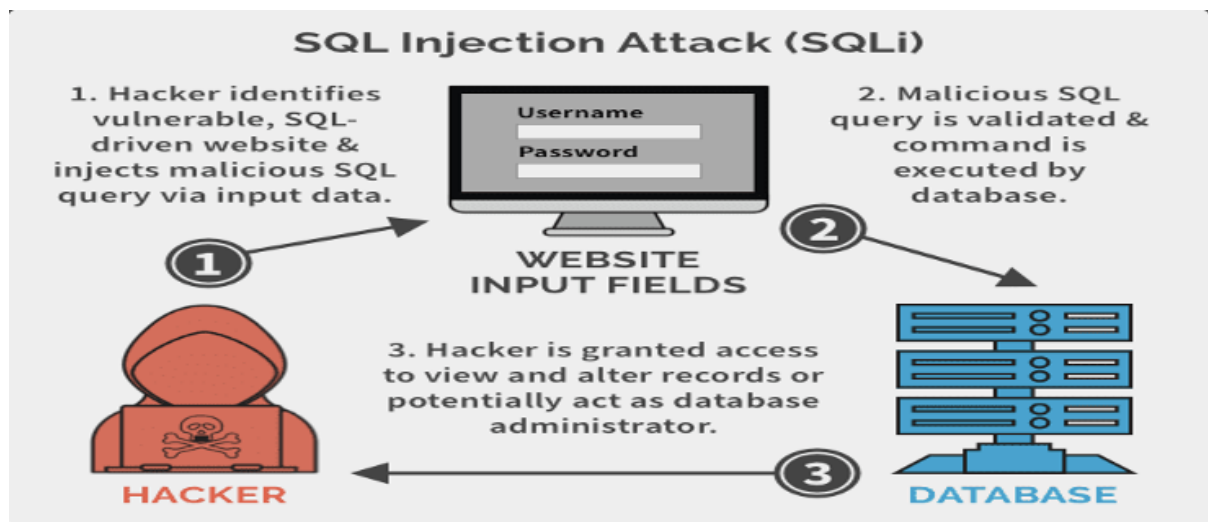


Figure 2: Overview of Injection Flaws

Typical injection vulnerabilities are:

- **SQL Injection (SQLi):** Attack that manipulates SQL queries to access or modify database data.
- **OS Command Injection:** Exploits vulnerable applications to execute system-level commands.
- **LDAP Injection:** Manipulates directory service queries to access or modify authentication data.
- **XML/XPath Injection:** Attackers manipulate XML or XPath queries to retrieve unauthorized data from XML-based systems.
- **NoSQL Injection:** Targets non-relational databases such as MongoDB by manipulating query operators to bypass authentication or extract data.

The failure to separate data from executable code allows attackers to manipulate queries using malicious input. Where user-provided input is simply included in commands, without sufficient validation, sanitization, or parameterization, the interpreter will not be able to differentiate between valid data and payloads controlled by an attacker. It is this separation of code and data that makes injection attacks so perilous and so prevalent in the past (PortSwigger, 2023).

1.3 Significance and Impact of SQL Injection

SQL Injection (SQLi) is one of the most severe security threats to web applications as it attacks the database layer where sensitive data is kept. Databases are used by many organizations to handle usernames, passwords, customer records, and payment details as well as business data. An effective SQL Injection attack can enable attackers to gain access to sensitive information, compromise logins, alter records or destroy valuable information (Halfond, 2006).

wallarm



Figure 3: CIA Triad

The effects of SQL Injection can be described in terms of CIA triad:

- Confidentiality: Disclosure of sensitive or private information.
- Integrity: Records that have been altered or deleted without authorization.
- Availability: Interruption of services because of broken databases.

Moreover, SQL Injection can lead to financial loss, reputation and legal penalties for organizations. SQL Injection remains a common threat despite modern security practices due to insecure coding and legacy systems (Clarke, 2012).

1.4 Technical Terminologies

To ensure clarity and precision throughout this report, the following key technical terms are defined:

Term	Definition
SQL (Structured Query Language)	A standardized language used to manage and manipulate relational databases.
Injection Attack	An attack where untrusted data is sent to an interpreter as part of a command or query.
Payload	The malicious data crafted by an attacker to exploit a vulnerability.
Input Validation	The process of verifying that user-provided data conforms to expected formats before processing.
Sanitization	Cleaning or filtering user input to remove or neutralize potentially harmful characters.
Parameterized Query	A database query that separates SQL code from user input using placeholders.
Blind SQL Injection	An SQLi technique where the attacker infers data based on application behavior rather than direct output.
DBMS	Database Management System — software that manages and provides access to a database.
OWASP	Open Web Application Security Project — a non-profit foundation providing standards for web application security.
Tautology	In SQLi, a condition crafted to always evaluate as true (e.g., 1=1).

Table 1: Technical Terminologies and Definitions

1.5 Aim and Objectives

Aim

The primary goal of this coursework is to critically examine the flaws of injections in web applications, mainly SQL Injection by conducting theoretical research, practical demonstration, and assessment of how to effectively tackle such weaknesses.

Objectives

- To describe the concept and the mechanism of injection flaws in web applications.
- To discuss the causes, types, and effects of the SQL Injection attacks.
- To identify and explain various types of injection vulnerabilities such as command and LDAP injection.
- To show SQL Injection attacks, in a controlled and ethical setting, using proper tools and methods.
- To assess mitigation measures like parameterized query, input validation and web application firewalls.
- To determine the strengths and weaknesses of the security controls chosen.
- To enhance the secure coding practice and raise awareness of the risks of web application security.

1.6 Report Structure

This report starts with the introduction section that gives an overview of injection flaws within web applications and specifically SQL Injection (SQLi). It discusses why web application security is important and the risks of injection attacks, including unauthorized access, data breach, and compromising the system. Other crucial technical terms are also defined in this section, and the purpose and objectives of the report are outlined.

The background section gives a comprehensive insight into injection vulnerabilities. It talks about the various forms of injection attacks, how they work, and typical causes like inadequate input validation and insecure query construction. Much focus is put on SQL Injection since it is more common and effective.

The demonstration section outlines how the security testing can be put into practice under a controlled setting. It describes how to set up OWASP Juice Shop and illustrates how various methods can be used to find and test vulnerabilities. Postman is also used to manipulate requests and OWASP ZAP is used to scan the vulnerabilities. Screenshots and analysis are used to support the results.

The mitigation part offers viable solutions to avoid injection attacks. The methods talked about are parameterized queries, input validation, sanitization, web application firewalls, least privilege, and secure error handling. The comparison provides critical analysis of both the strengths and limitations of the chosen mitigation strategies and details how they work in the real world.

Lastly, a conclusion summarizes the major findings of the report and reiterates that injection flaws are a major threat to web applications. It highlights the importance of secure coding practices, continuous testing, and security awareness to reduce future risks.

2. Background

2.1 Overview of Injection Flaws Vulnerabilities

One of the most significant security vulnerabilities of contemporary web applications are injection vulnerabilities. They are the result of the lack of proper validation of untrusted user input to an interpreter, like a database engine, shell of an operating system, or XML parser. This may enable attackers to modify application behavior, achieve unauthorized access, steal sensitive data or disrupt services (OWASP, 2021).

The primary reason behind faults in injecting is the inability to distinguish data and code. Where the user input is directly incorporated into the applications in terms of queries or commands, the malicious input can be executed as valid instructions. The issue still has numerous systems impacted by insecure coding, outdated software, and ineffective security testing (PortSwigger, 2023).

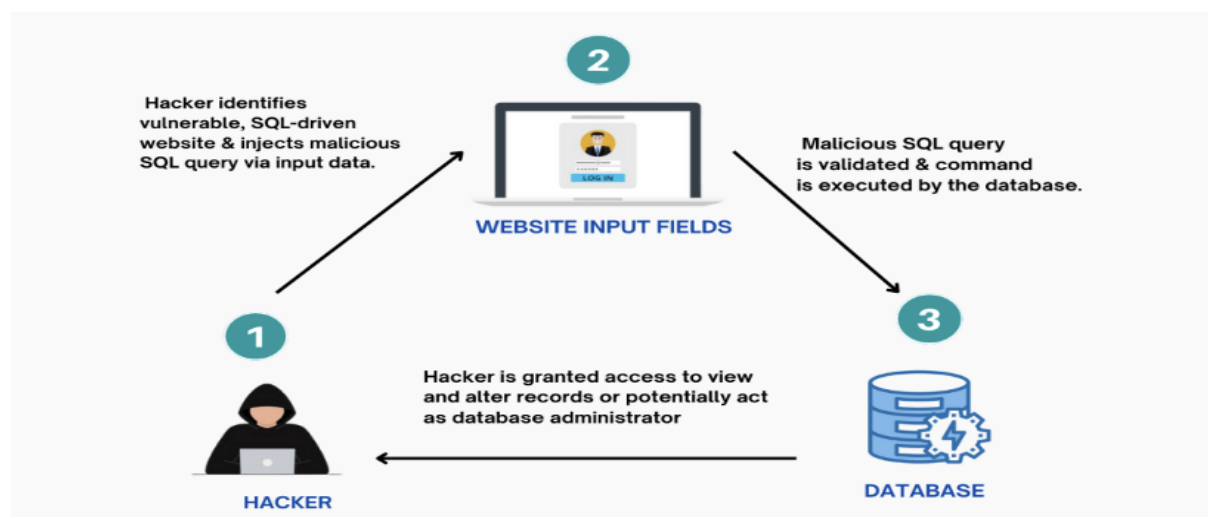


Figure 4: SQL Injection Mechanism

Typical injection vulnerabilities are:

- SQL Injection (SQLi): attacks relational databases.
- Command Injection runs operating system commands.
- LDAP Injection: exploits directory services.
- XML/XPath Injection: exploits XML queries.
- NoSQL Injection: attacks non-relational databases.

Of these, SQL Injection is one of the most prevalent and harmful as most web applications use databases to store sensitive data (Halfond, 2006).

2.2 Overview of SQL Injection

SQL Injection (SQLi) is a code injection attack, which attacks the database server of a web application. It arises when SQL statements are put into the application input maliciously and run by the backend database. This vulnerability usually occurs when web applications dynamically generate database queries using unsanitized user input without secure parameter handling (Halfond, 2006).

For example, a vulnerable login query may be written as:

```
SELECT * FROM users
WHERE username = 'admin'--'
AND password = '';
```

If an attacker enters the following payload into the username field:

```
admin'—
```

The resulting query becomes:

```
SELECT * FROM users
WHERE username = 'admin'--'
AND password = '';
```

The double-dash operator comments out the remaining query logic, which may allow attackers to bypass authentication mechanisms (PortSwigger, 2023).

Common SQL Injection Attack Process.

1. Attackers send malicious payload.
2. Payload is inserted into SQL query by Web application.
3. Database stores query which is manipulated.
4. The application produces unexpected behaviour or output.

2.3 Types of SQL Injection Attacks

SQL Injection attacks are often classified based on the way the data is retrieved and the reaction of the application.

2.3.1 In-Band SQL Injection

In-band SQLi relies on the same channel of communication to send payloads and receive the results.

- **Error-Based SQLi:** uses verbose database errors to divulge structure or syntax information.
- **Union-Based SQLi:** It is based on UNION SELECT statements to obtain information in other tables.

2.3.2 Inferential (Blind) SQL Injection

Blind SQLi is applied in cases where the query results are not directly shown by the application.

- Boolean-Based SQLi: monitors behaviors of pages depending on whether they are true or false.
- Time-Based SQLi: relies on delays deducing database responses like the use of SLEEP () to estimate response time.

2.3.3 Out-of-Band SQL Injection

In the absence of direct output, out-of-band SQLi employs alternate communication methods, like DNS or HTTP callbacks.

Type	Sub-Type	Detection Method	Example
In-Band	Error-Based	Error messages	SQL syntax errors
In-Band	Union-Based	Query results	UNION SELECT
Blind	TimeBased	Response difference	True/False logic
Blind	Time-Based	Delayed response	SLEEP()
Out-of-Band	DNS/HTTP	External callback	xp_dirtree

Table 2: Types and Detection Methods of SQL Injection Attacks

2.4 Causes of SQL Injection Vulnerabilities

The main factor that leads to SQL Injection vulnerabilities is the lack of security in the software development process. Common causes include:

- Inadequate input validation.
- String concatenation to construct queries dynamically.
- Not using parameterized queries.
- Lack of strong error management disclosing database information.
- Excessive database privileges
- Use of old or legacy systems.

These vulnerabilities are especially prevalent with older applications that have not been built into modern secure coding practices (OWASP, 2021).

2.5 Tools and Techniques for Identifying SQL Injection

There are numerous tools and techniques that security professionals can use to detect SQL Injection vulnerabilities in web applications. Modern tests tend to use automated scanning, controlled exploitation, and manual request testing to cover all the areas rather than merely relying on interception-based platforms.

In this report, the focus is on tools that aid in vulnerability discovery as well as ethical testing in a controlled laboratory setting.

Commonly Used Tools

OWASP ZAP: An open-source security testing framework that is utilized to detect vulnerabilities by automated testing, passive and spidering of web applications (ZAP, 2026).

Sqlmap: Sqlmap is an automated penetration testing tool commonly used to identify and evaluate SQL Injection vulnerabilities in web applications (sqlmap, 2026).

Postman: Postman is an application that facilitates the construction and manipulation of HTTP requests manually, which enables testers to inject payloads into parameters, headers, and request bodies, and examine responses (Postman, 2026).

Techniques Used for Detection

- OWASP ZAP automated vulnerability scanning.

- Manual payload injection and request modification with Postman.
- Automated exploitation and testing with sqlmap.
- Input fuzzing to find sensitive parameters and non-regular responses.
- Comparison testing of the responses and error analysis.

These tools and techniques offer both manual as well as automated ways of identifying SQL Injection vulnerabilities and upholding ethical and authorized testing limits.

2.6 Consequences of SQL Injection

SQL Injection attacks have the potential to cause serious impacts on the confidentiality, integrity and availability of systems.

- **Confidentiality:** attackers can gain access to usernames, passwords, payment information or customer records.
- **Integrity:** attackers can alter, add or remove information stored.
- **Availability:** attacks can corrupt databases or cause disruption to the application services.

Also, organizations can lose money, reputation, regulatory fines and court cases (OWASP, 2021).

2.7 Real-World Incidents of SQL Injection

SQL Injection vulnerabilities have been involved in several key cyber-attacks.

- In 2015, TalkTalk was victimized by a big breach when attackers capitalized on SQL Injection vulnerabilities to reveal customer data.
- Heartland Payment Systems was the victim of a breach connected to the compromise of databases, which demonstrates vulnerabilities to financial systems.

These attacks show that insecure web applications can pose serious business and legal impact.

2.8 Penetration Testing Execution Standard (PTES)

The Penetration Testing Execution Standard (PTES) offers an organized system of approved security testing.

1. Pre-Engagement: specify scope, objectives and permissions.
2. Intelligence Gathering: locate targets, endpoints and inputs.

3. Threat Modelling: establish probable attack vectors.
4. Vulnerability Analysis: SQL Injection point test.
5. Exploitation: Out controlled proof-of-concept attacks.
6. Post-Exploitation: evaluating impact and exposure to privilege.
7. Reporting: record findings and remediation recommendations.

PTES ensures uniformity, legalization and professionalism in security tests.

2.9 Evolving Attack Methods

Attackers still evolve more sophisticated SQL Injection, such as:

- Coded or encrypted payloads.
- Bypass methods of Web Application Firewall.
- Second-order SQL Injection
- SQLi with XSS.
- In modern applications, API parameter injections are in use.

This shows that there should be constant defensive enhancement.

2.10 Modern Defence Strategies

When properly implemented, modern defense strategies will minimize SQL Injection risk.

- Prepared statements / parameterized queries.
- Good input validation and sanitization.
- Web Application Firewalls (WAFs)
- Minimized access to databases.
- Secure coding standards
- Regular penetration testing
- Continuous vulnerability scanning

Nevertheless, no one's control alone is adequate. Layered security controls, secure development practices, and constant monitoring are needed to ensure effective protection.

3. Demonstration of Injection Flaws in Web Applications Using OWASP Juice Shop

3.1 Introduction to the Practical Demonstration

To support the theoretical insights outlined in Section 2, we conducted a practical security analysis using OWASP Juice Shop, a vulnerable web application provided by the Open Web Application Security Project. Juice Shop was designed as a platform for cybersecurity training, penetration testing and security awareness in safe and legitimate contexts (OWASP, 2023). The app is deliberately vulnerable to a range of security issues mapped to the OWASP Top 10 risks, providing an industry-standard platform for security demonstrations.

Injection vulnerabilities remain among the most critical security threats affecting modern web applications. Recent research shows that despite being well-known for decades, a lack of input sanitization and secure query building still leaves web applications vulnerable to exploitation (Neupane, 2025). This practical exercise sought to illustrate the process of identifying and exploiting insecure authentication processes using injection attack payloads, capturing and inspecting API communication in the back end, and using a mix of both manual and automated tools to perform a complete security assessment of web applications.

This exercise follows the Penetration Testing Execution Standard (PTES) principles outlined in Section 2.8, following a phased approach to reconnaissance, vulnerability analysis, exploitation, post exploitation assessment and impact evaluation (PTES, 2014). The testing was limited to within a localhost laboratory environment on a Kali Linux system. No third-party or production systems were accessed. Throughout the testing process, ethical consideration was maintained by only accessing the intentionally vulnerable lab environment.

3.2 Practical Laboratory Environment Setup

The testing environment was set up using Kali Linux, a Debian-derived Linux distribution designed for penetration testing, security research and digital forensics, used by security professionals around the world (Security, 2024). The OWASP Juice Shop application was run with the use of Docker, a tool for containerizing applications that provide an isolated, consistent and reproducible environment for the running applications, independent of the host operating system (Markel, 2014). The

containerized deployment ensured the stability and isolation of the testing environment.

The following commands were used:

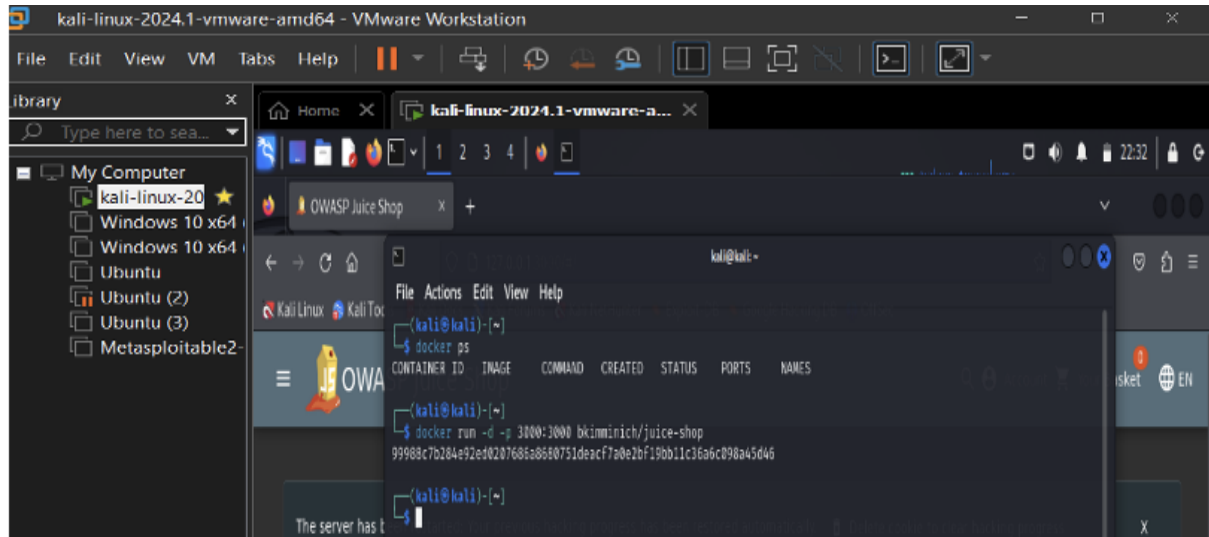


Figure 5: OWASP Juice Shop Running on Kali Linux

The application was accessible at <http://127.0.0.1:3000/>

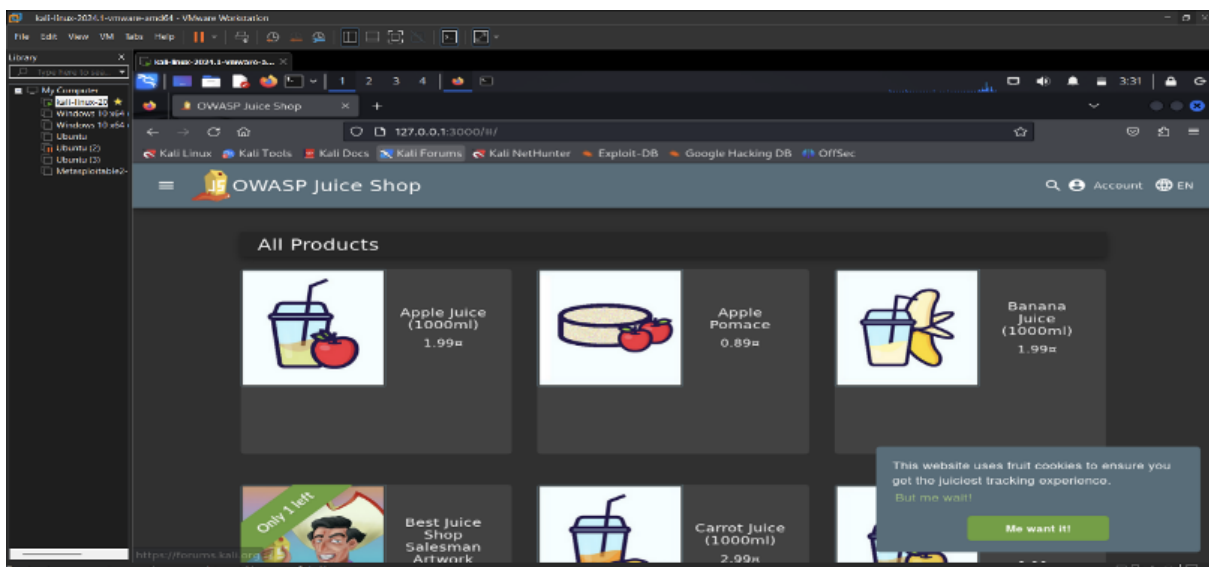


Figure 6: OWASP Juice Shop Running at Host

Tools	Purpose
OWASP Juice Shop	Intentionally vulnerable target application
Kali Linux	Security-focused testing operating system
Docker	Containerized application deployment
Firefox and Developer Tools	Browser interaction and traffic inspection
Postman	Manual API request construction and testing
Sqlmap	Automated SQL Injection detection and testing
OWASP ZAP	Automated web application vulnerability scanning
Jwt.io	JWT token decoding and analysis

Table 3: Tools and Their Purpose in the SQL Injection Testing Environment

3.3 Reconnaissance and Target Identification

The process of reconnaissance is dedicated to discovering opportunities for attack within the target system (PTES, 2014). Authentication services are common targets as they handle direct user-supplied credentials, which are often susceptible to injection-style attacks when input is not properly handled (PortSwigger, 2023). Indeed, research has shown that authentication is one of the most vulnerable entry points to attack in web applications in that it is connected to the underlying database systems (Sherekar, 2022).

POST /rest/user/login

Using the Firefox Developer Tools I checked the network traffic. It showed that the login form talks to this API endpoint:

Passwords were sent to the server in JSON, confirming user input was processed at the API level and not client side (Stuttard, 2011).

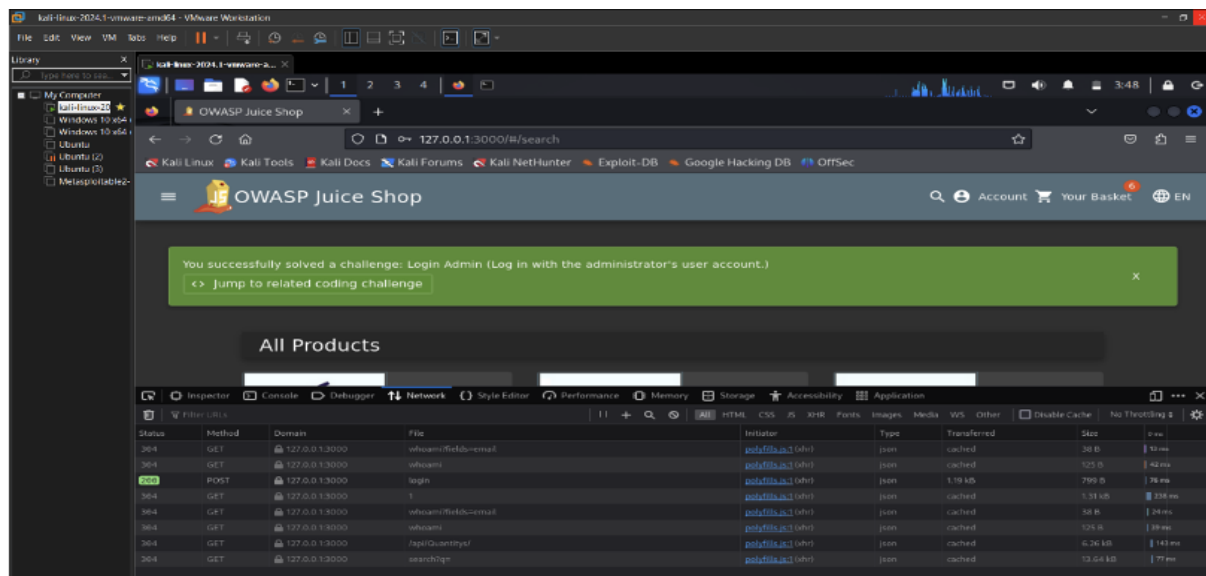


Figure 7: Identification of the /rest/user/login API endpoint using browser Developer Tools

3.4 Baseline Authentication Testing

The application was tested using standard accounts prior to the security testing to establish the baseline behavior. Baseline testing is considered essential in professional penetration testing as it provides documentation of the normal behavior of an application, facilitating the detection of abnormal behavior during the attack simulation (Weidman, 2014).

Email: test@test.com

Password: test123

The application failed authentication as expected, indicating that it was operating properly before the security testing began.

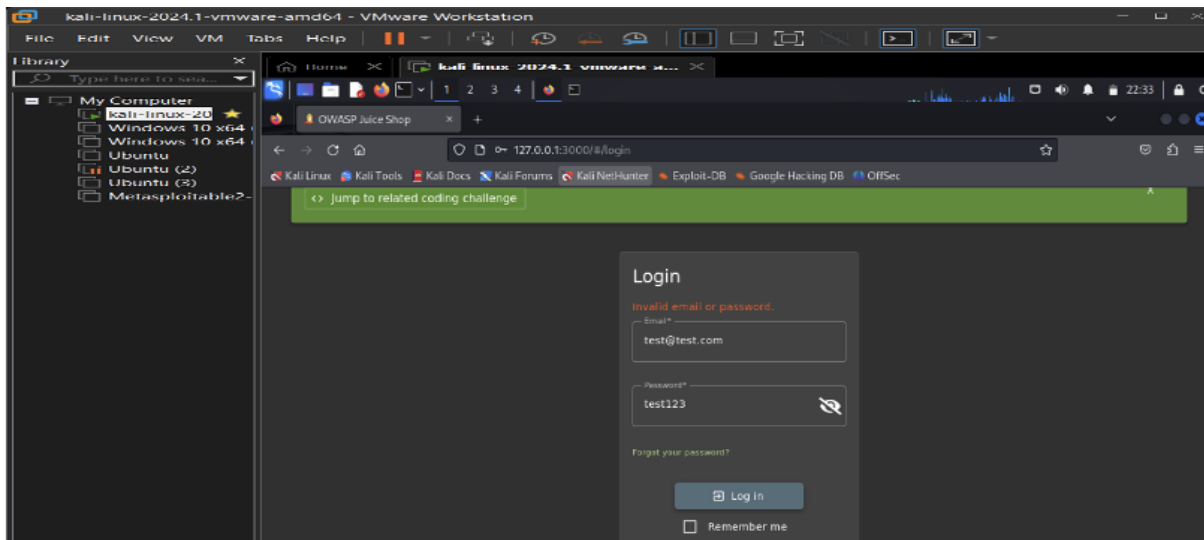


Figure 8: Baseline authentication test showing invalid credentials response

3.5 Payload Preparation and Technical Analysis

A tautology-based injection payload was constructed to test the authentication system. A tautology attack injects a condition that is always true into an SQL statement, circumventing authentication when queries are dynamically constructed using insecure string concatenation rather than parameterized queries (Halfond, 2006). Studies show that tautology injections continue to be a common and successful attack on web application authentication mechanisms (Sherekar, 2022).

The payload constructed was:

Email field: ' OR 1=1--

Password field: test123

This payload represents an injection-style authentication bypass attempt using tautology-based logic manipulation. The predicate $1=1$ is always true, so authentication logic can be circumvented in case input validation is weak. The two dashes (--) turn off the rest of the query, including the password verification.

Although the payload resembles a traditional SQL Injection attack, the observed behaviour within OWASP Juice Shop is more accurately categorized as an injection-style authentication bypass caused by insecure backend input handling and weak API validation. Therefore, the observed behaviour is more accurately categorized as an

injection-style authentication bypass rather than direct SQL query manipulation (Halfond, 2006).

```
SELECT * FROM Users WHERE email=" OR 1=1--' AND  
password='test123'
```

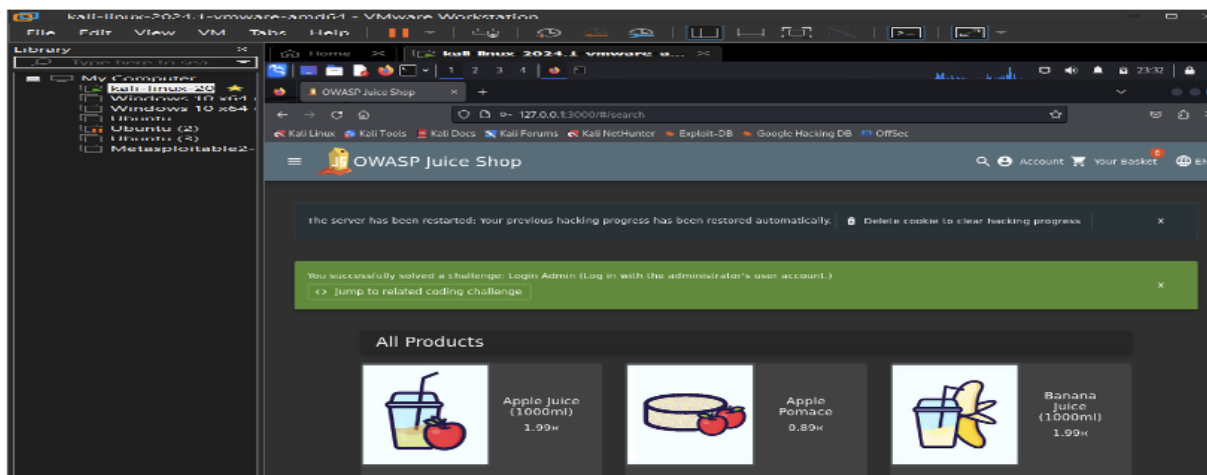


Figure 9: SQL Injection Payload Prepared for Authentication Bypass Testing

3.6 Exploitation and Authentication Bypass

Upon submitting the payload, the application returned a response indicating successful authentication behavior. The system granted access to an administrator account and triggered a challenge completion message.

This result demonstrates insufficient backend validation and insecure processing of authentication requests. The observed behavior is consistent with an injection-style vulnerability caused by improper input validation rather than classical SQL query execution (Stuttard, 2011).

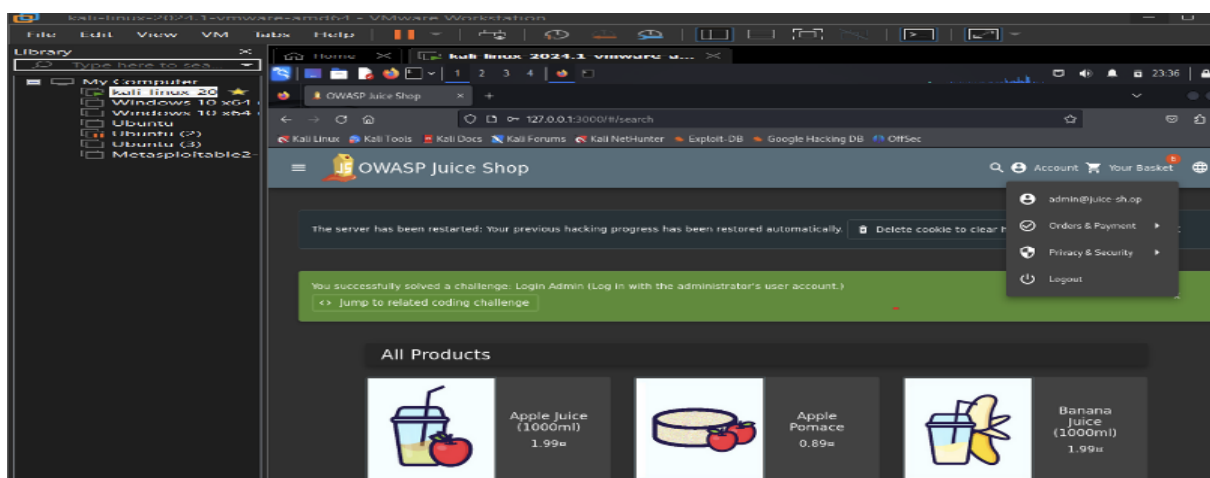


Figure 10: Successful authentication bypass using injection-style payload

3.7 HTTP Request and API Layer Analysis

The login request was examined in Developer Tools to confirm the API layer behaviour.

Property	Observed Value
HTTP Method	POST
Endpoint	/rest/user/login
Content-Type	Application/json
Request Type	XHR / API Request
Response Status	200 OK

Table 4: Observed HTTP Request Properties for the Login Endpoint

The raw body of the request revealed that the injected code was sent to the API unfiltered, without sanitization or parameterization before it was used to build a query (Clarke, 2012).

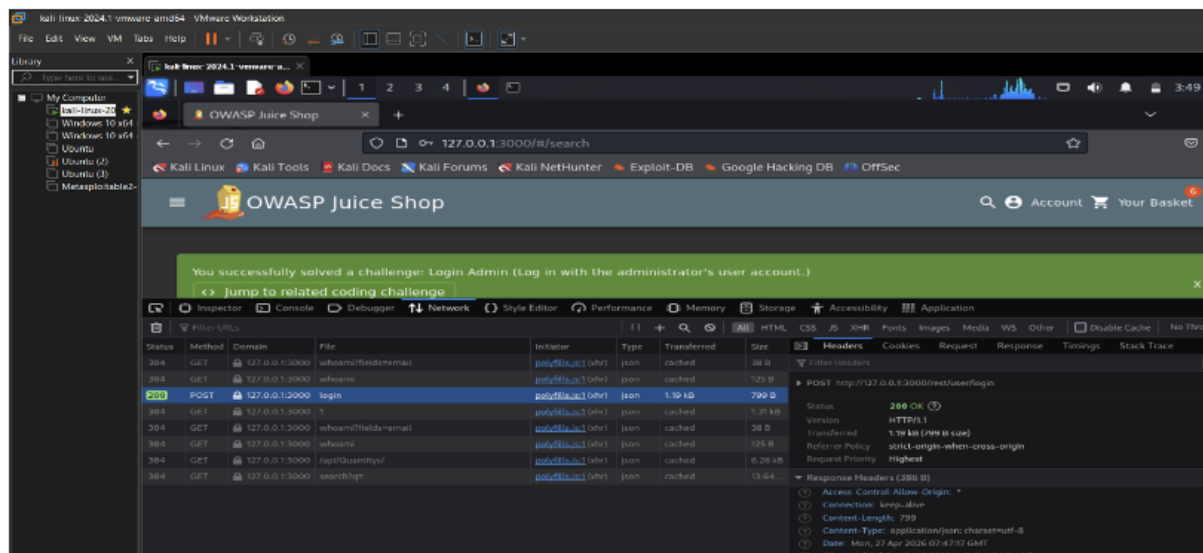


Figure 11: Captured HTTP POST Login Request Containing Injection Payload

3.8 Token Analysis and Session Validation

After the bypass, the local storage of the browser was checked and contained a JSON Web Token (JWT), which means the server released a valid authenticated session token. JWT (JSON Web Token) is an open standard as defined in RFC 7519 for securely passing authentication information between parties (Michael Jones, 2015). But recent studies stress that the security of JWT is reliant on correct implementation and that insecurely configured tokens are key to creating serious vulnerability in the modern web application (A. Bucko, 2023).

The token was decoded with jwt.io and displayed administrator account claims, email address of `admin@juice-sh.op`, the internal ID of an administrator account and the current session times. This indicated the bypass resulted in a server-backed, fully authenticated, administrator account session and not a cosmetic disguise (OWASP, 2021).

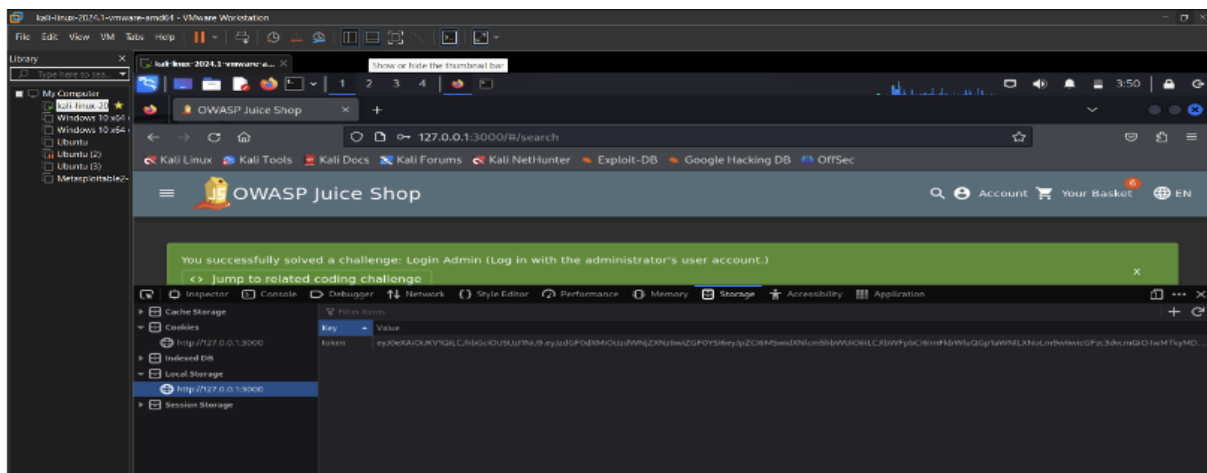


Figure 12: JWT Token Returned after Successful Authentication Bypass

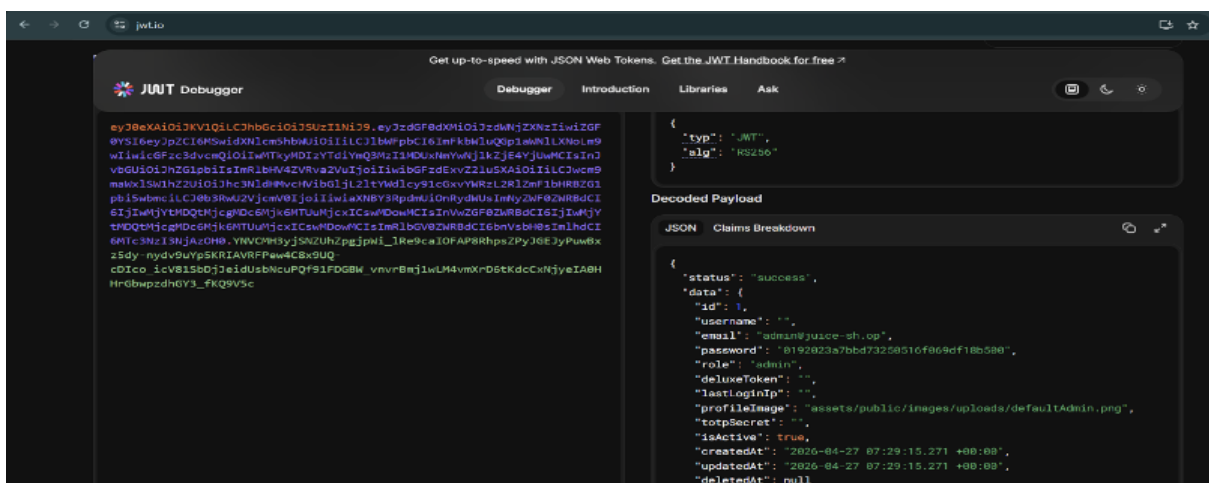


Figure 13: Decoded JWT Token Showing Administrator Session Claims

3.9 Validation of Authenticated Requests

This verifies that the application uses token-based authentication for all operations with privileged information after a successful login (Jones, 2015). Bucko et al. (2023) highlight that JSON Web Tokens (JWT) stolen via network sniffing or cross-site scripting attacks can allow full session hijacking without the need to re-submit the original payload, increasing the potential impact of token-based authentication vulnerabilities.

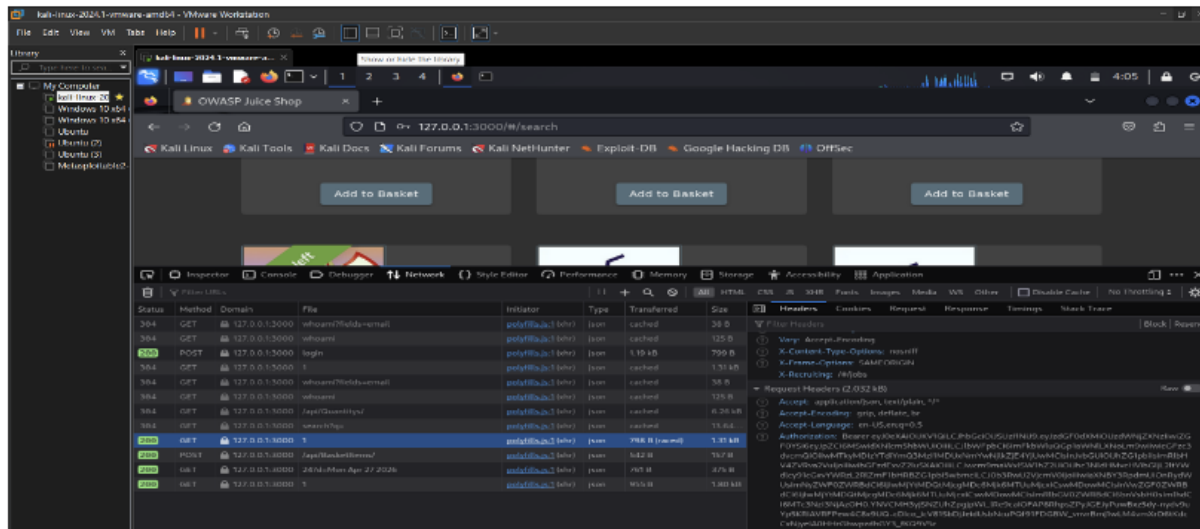


Figure 14: Authenticated API Request using JWT Bearer Token

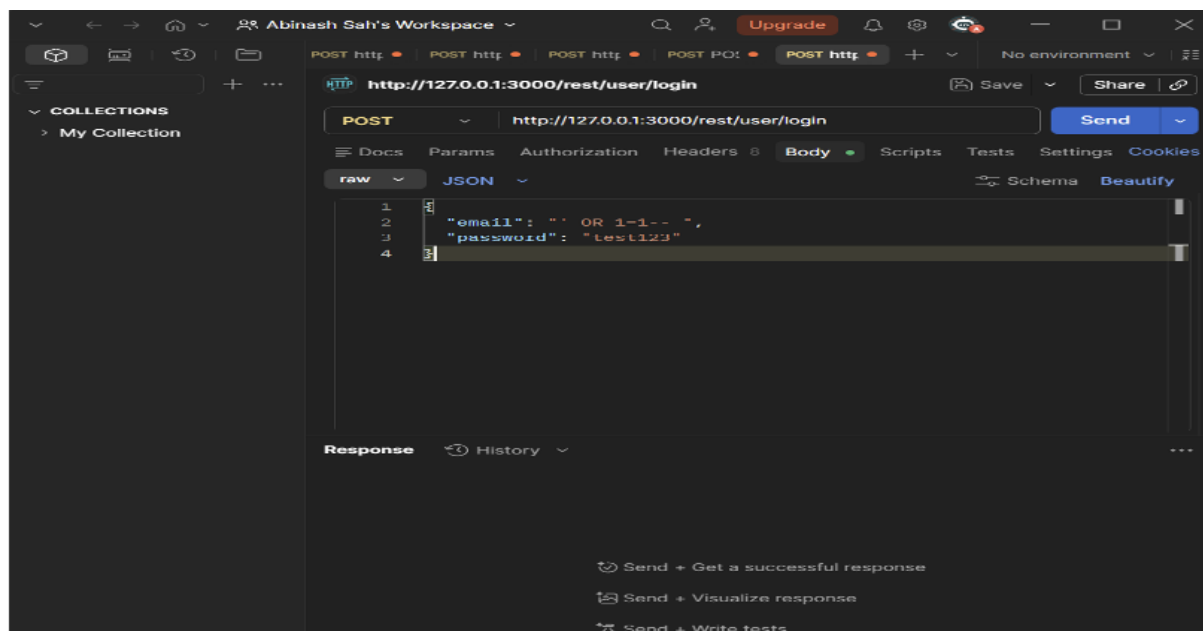


Figure 15: Manual API Testing in Postman Demonstrating

3.10 Manual API Testing using Postman

To reproduce the vulnerability outside of the browser, the login request was reproduced in Postman, a tool that enables direct requests to the backend services, bypassing all client-side checks (Postman, 2024).

The response from the server was HTTP 200 OK, with a token and confirmation of the `admin@juice-sh.op` account. This independently confirms the vulnerability is in the backend API and can be exploited without launching a browser. As recent penetration testing research confirms (Neupane, 2025), client-side controls provide no effective barriers to prevent attacks against backend APIs using tools like Postman or custom scripts (Stuttard, 2011).

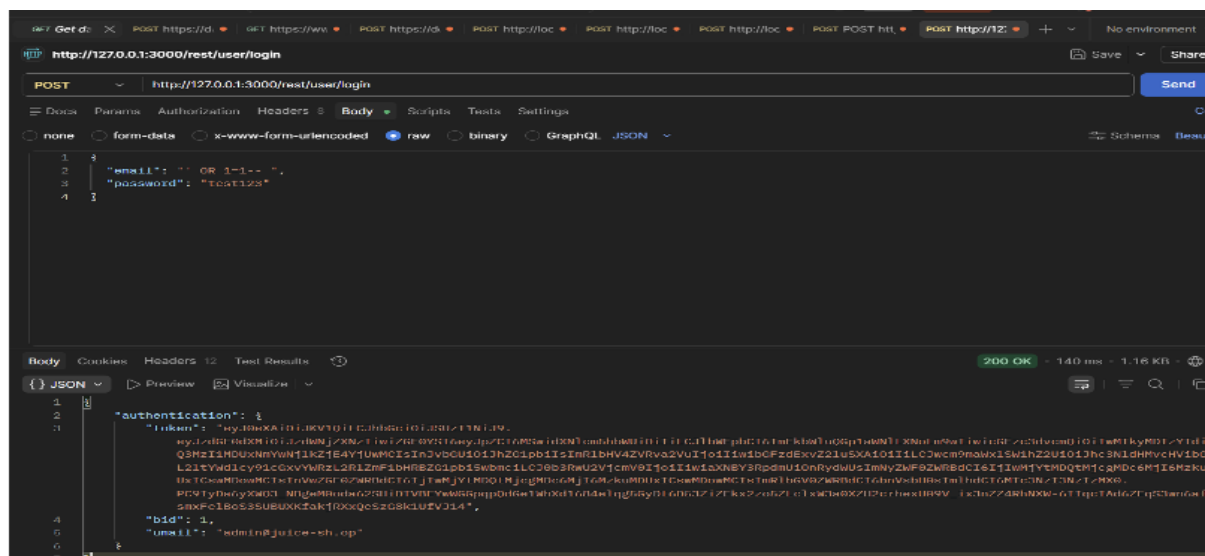


Figure 16: Manual API Testing in Postman Demonstrating Successful Authentication Bypass

3.11 Automated Testing using sqlmap

The process of automated testing was performed with the help of the sqlmap open-source tool, which is a dedicated tool that helps to identify and exploit SQL Injection vulnerabilities in various database systems (sqlmap, 2026).

The request obtained was fed to the sqlmap to be analyzed. The app responded to all automated probes with the same HTTP 401 Unauthorized reply. This was since the payloads of SQL map failed to replicate the exact format of the JSON and tautology pattern needed, and some application frameworks do not lend themselves to generic automated injection patterns but can be susceptible to a well-crafted manual payload (Clarke, 2012).

A classical SQL Injection vulnerability was not identified using the sqlmap tool. The reason behind this is the application uses structured API inputs based on JSON, instead of the conventional SQL query parameters to conduct the authentication of a request.

The findings demonstrate that certain logic-based vulnerabilities may not be reliably identified through automated scanning alone. Thus, manual testing is still critical in the detection of complex vulnerabilities of web applications in the contemporary world (sqlmap, 2026).

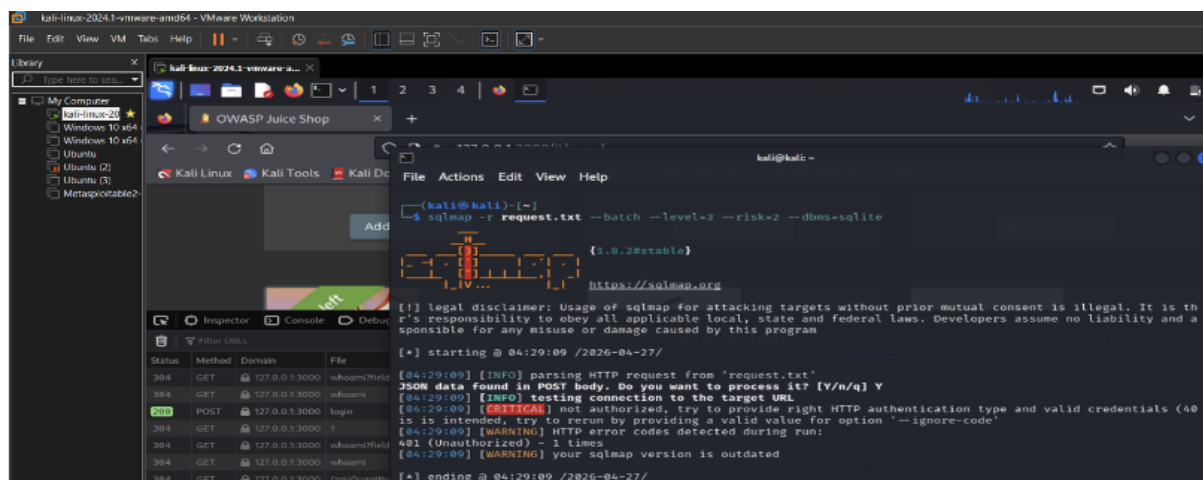


Figure 17: Automated SQL Injection Testing using sqlmap

3.12 Defensive Assessment using OWASP ZAP

A passive and active scan was performed automatically with the help of OWASP ZAP which is a popular open-source web application security scanner (OWASP ZAP, 2024). Recent studies have validated the fact that OWASP ZAP, when used in combination with manual testing, delivers much more vulnerability coverage than automated scanning (Neupane, 2025).

It is interesting to note that the SQL Injection bypass in Section 3.6 was not specifically identified by OWASP ZAP. This was possible since the bypass takes advantage of manipulation of application-level query logic as opposed to generating a standard injection error signature which can be detected using automated pattern matching (Stuttard, 2011). This supports the need to integrate automated scanning and competent manual penetration testing during all security testing.

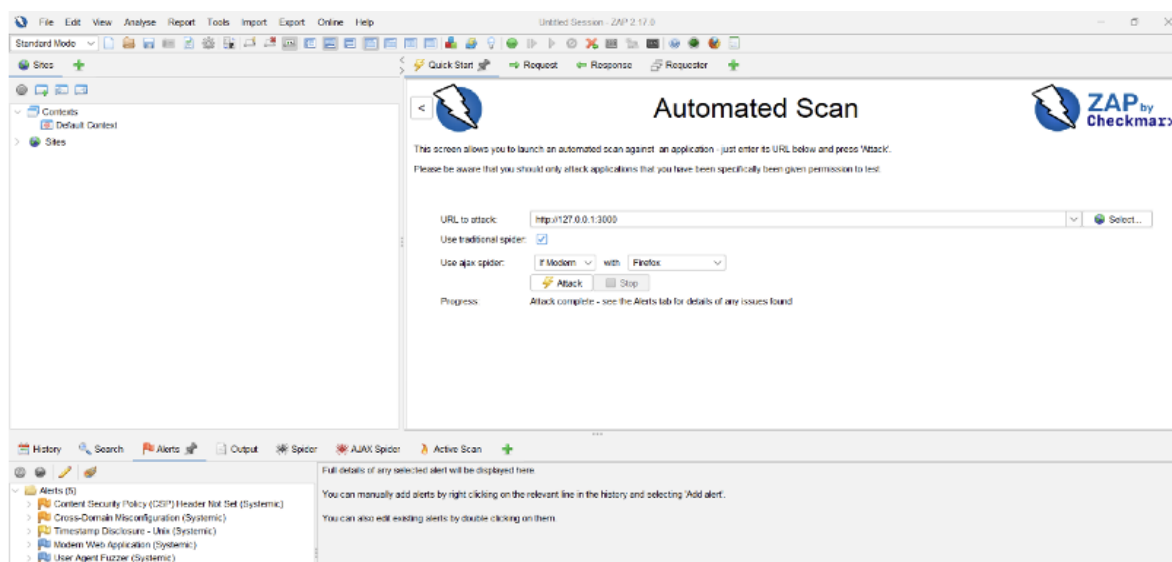


Figure 18: OWASP ZAP Vulnerability Scanning Results

3.13 Critical Evaluation of the Demonstration

This experiment shows that modern web apps are not necessarily susceptible to classical SQL Injection but may be vulnerable to injection-like vulnerabilities because of lax input validation and insecure backend logic. The tautology-based payload that is used evidence how attackers can be able to affect authentication operations even when direct SQL execution is not apparent.

More so, automated applications like SQLmap or OWASP ZAP failed to identify this behaviour in totality, which proves that manual testing is still a crucial part of detection of complex vulnerabilities.

Thus, good practices in secure coding should involve robust input validation, parameterized queries and secure API design to avoid both traditional and modern injection attacks (Neupane, 2025).

The findings highlight that modern web application security requires both automated vulnerability assessment and skilled manual penetration testing to accurately identify complex injection-related vulnerabilities (Markel, 2014).

4. Mitigation of Injection Flaws in Web Applications

4.1 Prepared Statements / Parameterized Queries

The best way to avoid injection attacks is to use parameterized queries (prepared statements). This helps to ensure that user input is taken as data, not as code to execute. This means that the malicious payloads like ' OR 1=1--' can no longer change the query's intended logic since they are separated from the values provided by the user (OWASP, 2023).

Secure applications for instance, use placeholders and bind variables in their SQL instead of dynamically assembling queries. In this way, intruders cannot add more instructions to the queries sent to the database. Modern frameworks and Object-Relational Mapping (ORM) tools also natively support parameterized queries, which further mitigates the risk of vulnerability from injection attacks (Oracle, 2024).

4.2 Input Validation and Sanitization

Input validation is a basic security control that restricts the data that can be entered into the application to only expected and safe data. All user data should be strictly checked on server side including data type, length and format.

Also, sanitization methods can be used to either remove or encode potentially dangerous characters. For instance, characters like ', " and special operators for injection attacks need to be dealt with appropriately. But validation is not enough and should be used with other security methods like parameterized queries (PortSwigger, 2023).

4.3 Web Application Firewall (WAF) – Practical Implementation

To showcase mitigation in practice, a Web Application Firewall (WAF) was deployed, created with the Python language and Flask Web Application Framework, a lightweight proxy. The WAF sits in front of the OWASP Juice Shop application, and processes the incoming HTTP requests, performing checks for it before sending them to the back server (Projects, 2024).

Implementation Setup

- Backend application (OWASP Juice Shop) running on:
<http://127.0.0.1:3000>
- WAF proxy running on:

<http://192.168.163.128:8080>

The WAF was configured to detect common injection patterns such as:

- ' OR 1=1—
- SQL keywords (SELECT, UNION, DROP)
- Suspicious symbols and logical operators

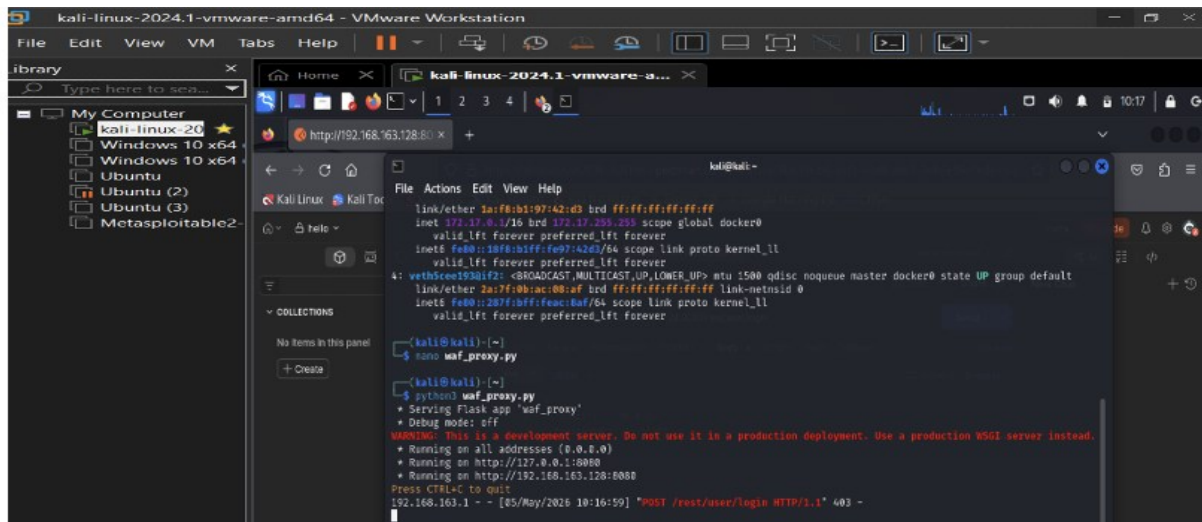


Figure 19: Python Flask Web Application Firewall Running on Port 8080

4.3.1 Normal Request Behaviour

A typical login request was submitted to the WAF with valid input:

```
{  
  "email": "test@test.com",  
  "password": "test123"  
}
```

The request was successfully forwarded to the backend application by the WAF. The server returned a 401 Unauthorized response because invalid login credentials were supplied. This confirms that the WAF does not block legitimate application traffic and that common applications don't lose out functionality.

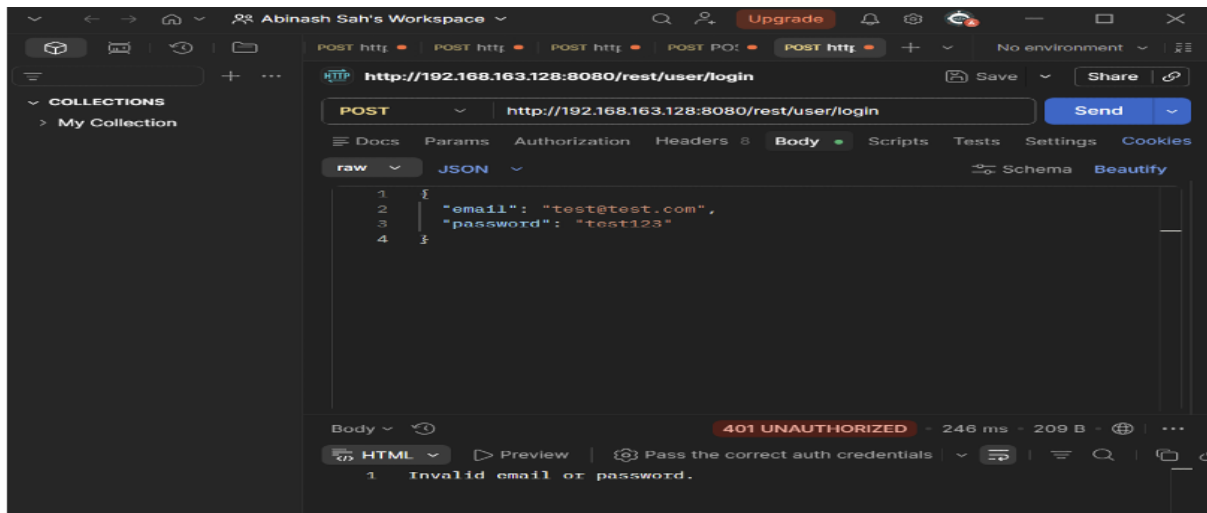


Figure 20: Legitimate Login Request Allowed through WAF Returning 401 Unauthorized

4.3.2 Attack Request Behaviour (Blocked)

A malicious payload was then submitted:

```
{  
  "email": "\" OR 1=1--",  
  "password": "test123"  
}
```

The WAF detected the injection pattern and blocked the request before it reached the backend application. The response returned was:

403 Forbidden – Malicious request blocked by WAF

This demonstrates that the WAF effectively prevents injection attacks by filtering malicious input at the application layer (OWASP, 2021).

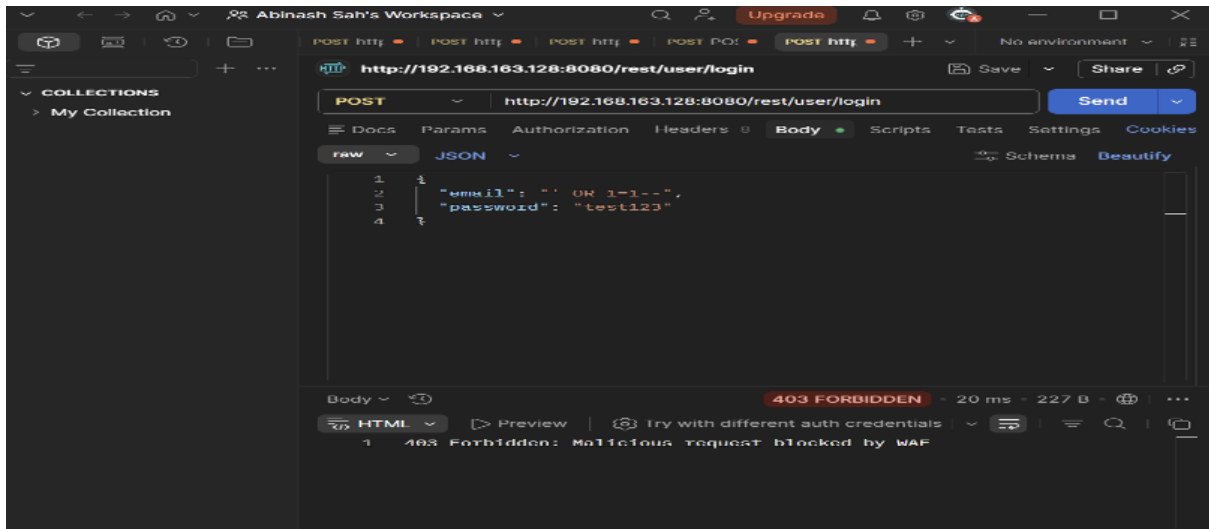


Figure 23: Injection Payload Blocked after WAF Implementation

4.4 Least Privilege for Database Accounts

One of the other key mitigation measures is the principle of least privilege. Applications should have the least privileges necessary to perform their functions in the database. For instance, access to the deletion of tables, or changes to the database schema should not be granted to a login system (NIST, 2022).

With restrictions on privileges, even if an attacker is successful exploiting an injection vulnerability, the potential damage is greatly reduced.

4.5 Proper Error Handling

Applications should not reveal error messages to the user – these can be helpful for attackers. For instance, if there are any errors in your database that show the structure of query or information about systems, it can help you make a more effective injection attack (OWASP, 2023).

Rather, their errors should be presented as user-friendly error messages, and their logs should be securely stored to be analyzed administratively.

4.6 Secure API Design

The `/rest/user/login` endpoint of the application demonstrates that APIs are heavily used in modern Web applications for communication between clients and back-end services. As a result, API security is a crucial aspect of web application security.

Best practices for securing APIs include:

- Server-side JSON validation
- Denying/ignoring unexpected fields and operators
- Robust authentication and authorization
- Schema validation and secure input filtering mechanisms
- Rate limiting and access control

While it may not be possible to exploit traditional SQL Injection, these vulnerabilities can be introduced when APIs are improperly validated. Manipulating requests, bypassing security controls, or sending malicious payloads are vulnerabilities of poorly designed APIs (OWASP, 2021).

4.7 Regular Security Testing and Monitoring

Ongoing security testing will help determine where vulnerabilities may be present so that they can be fixed before they are taken advantage of by an attacker. Companies should have automated and manual security testing processes throughout the software development lifecycle.

Recommended approaches include:

- Manual penetration testing
- Auto-scan vulnerabilities with tools like SQLmap, OWASP ZAP
- Secure code reviews
- Vulnerability assessment and patch management
- Monitoring and logging on an ongoing basis

Whereas, as shown in Section 3, automated tools are not always able to detect all vulnerabilities, especially those based on logic injections. Thus, manual and automated testing can be used together to offer a wider coverage and to improve the accuracy of vulnerability detection (Weidman, 2014).

4.8 Strengths and Limitations of Mitigation Techniques

Technique	Strength	Limitation
Parameterized Queries	Strong protection against SQL Injection	Requires correct implementation
Input Validation	Prevents malicious input	Can be bypassed if poorly designed
WAF	Filters known attack patterns	Not detecting all custom attacks
API Security	Protects modern architectures	Complex to implement correctly
Security Testing	Identifies vulnerabilities	Requires expertise and time

Table 5: Strengths and Limitations of SQL Injection Mitigation Techniques

5. Evaluation

5.1 Prepared Statement / Parameterized Queries

Below are discussed the advantages of mitigation strategies using prepared statements / parameterized queries:

- Prepared statements isolate SQL code from what users' type, which prevents SQL Injection attacks. User supplied data is used only as input data and is not passed as part of an SQL command, thus minimizing the risk of injection attacks.
- They ensure reliable protection on several programming platforms, including PHP, Java, Python, .NET, ensuring consistent security implementation in different development environments.
- By using prepared statements, developers are more likely to ensure that they construct queries safely, avoiding unsafe queries. This will make the code more maintainable, reliable and less prone to future vulnerabilities.
- There are plenty of modern development frameworks and database libraries that support parameterized queries; they are not only easy to use but also popular and industry-standard security measures in modern web applications.
- Major cybersecurity groups, like OWASP and the National Institute of Standards and Technology (NIST), strongly recommend prepared statements as a key line of defense against SQL Injection attacks (NIST, 2022).

The disadvantages of prepared statements / parameterized queries as mitigation strategies are discussed below:

- Parameterized queries can be difficult to implement in legacy systems, with extensive redevelopment and refactoring of the code, which can be time consuming and costly.
- The use of prepared statements could lead to vulnerabilities if the developers are not knowledgeable about secure coding practices.
- But if the parameterized queries are not fully implemented, then the vulnerable portion of the application may be left exposed to injection attacks.
- Many more complicated queries might simply not be possible to translate to a fully parameterized equivalent, especially in older applications with not well-structured codebases.

- Additional developer training and secure coding awareness programmers may be needed for the proper and consistent implementation of the organizations (Oracle, 2024).

Below are shown the areas of application for prepared statements / parameterized queries as mitigation strategies:

- **New System Development:** Highly effective if they are used for newly developed web applications, where it is possible to establish good coding practices from the start of the software development lifecycle.
- **Refactoring of the Legacy System:** It is often used to upgrade old systems to meet new security requirements and secure software development practices.
- **Multi-Platform Applications:** Works on applications running on various programming platforms (PHP, Java, Python, .NET), with security controls in the database that are consistent.
- **High Security Systems:** Designed for applications that involve highly sensitive information such as:
 - Banking systems
 - Healthcare applications
 - E-commerce platforms
 - Government systems
 - Military and defense applications
- **Enterprise Web Applications:** These are typically deployed in a modern enterprise environment where large amounts of confidential data are stored in databases, such as data from customers and other organizational data (OWASP, 2025).

5.2 Input Validation and Sanitization

Below are some of the benefits of input validation and sanitization as a mitigation measure:

- Validates the input to the application to ensure that only valid and expected data can be accepted.
- Prevents malicious payloads from SQL Injections and other attacks.
- Minimizes risks of unauthorized access and alteration of information.

- Enhances the reliability and security of the application.
- The practice of using a good secure coding technique, as recommended by (OWASP, 2023).

The disadvantages of input validation and sanitization as mitigation strategies are mentioned below:

- Advanced attackers can circumvent weak validation rules.
- Input validation alone cannot fully eliminate injection vulnerabilities.
- Bad implementation could prevent legitimate users from inputting data.
- For complex applications, you need lots of validation logic.

Below are mentioned some of the areas where input validation/sanitization is used as a mitigation technique:

- Login and authentication systems
- Web forms and search boxes
- API request validation
- e-commerce and banking applications
- Enterprise and cloud-based systems

5.3 Web Application Firewall (WAF)

Below are listed the benefits of using Web Application Firewalls (WAFs) as mitigation measures:

- WAFs can block malicious HTTP requests before they enter the Web application using filters to help prevent injection attacks.
- They put an extra defense against SQL Injection, Cross-Site Scripting (XSS) and other web-based attacks.
- WAFs can be used to secure legacy applications with no changes to the application source code.
- They assist in keeping track of unusual traffic and recognize attempted attacks as they occur.
- The WAF implemented in this coursework successfully detected and blocked the malicious payload ' OR 1=1-- from the web server and returned a 403 Forbidden response.

- According to OWASP, WAFs are an important defense-in-depth security mechanism (OWASP, 2021).

The Web Application Firewalls (WAFs) are discussed as disadvantages below in terms of their use as mitigation strategies:

- Advanced or obfuscated payloads can evade simple, rule-based WAFs.
- If not configured correctly, it can result in false alarms and prevent valid users from accessing the system.
- WAFs cannot provide complete protection of insecure applications.
- An attacker with skill could circumvent the weak filtering by encoding or by using logic-based attacks.
- Technical knowledge is needed for proper setting up and maintenance.

The following are listed as some of the areas where Web Application Firewalls (WAFs) are used as a mitigation measure:

- Applications that are web accessible and used by Internet users
- E-commerce and online banking systems
- Business portals & APIs
- Legacy applications requiring additional protection
- Cloud-based web services and the modern API architecture (OWASP, 2021).

5.4 Least Privilege for Database Accounts

The following describes some of the benefits of least privilege for database accounts as mitigation strategies:

- Restricts access to the database to the least privilege needed for the application.
- Minimizes the consequences of effective injection attacks and unauthorized access.
- Does not allow attackers to carry out high level administrative operations like deleting tables or altering databases.
- Reduces lateral movement within systems following compromise.
- Advised by NIST as a key principle in secure system design (NIST, 2022).

Below are listed the drawbacks of least privilege for database accounts as countermeasures:

- In a large system, permissions and user roles can be tricky to manage.
- Privilege settings can impact the regular use of applications.
- Needs ongoing supervision and control.
- Implementation can add to the overhead in enterprise environments.

Below are listed some of the areas of least privilege for database accounts as mitigation strategies:

- Financial and bank systems.
- Patient record and healthcare systems
- Enterprise database applications
- Government and military systems
- Cloud-based and large-scale web applications

5.5 Proper Error Handling

Below are listed the benefits of effective error handling as means to mitigate:

- Stops exposing sensitive information from a system to an attacker.
- Minimizes the potential for attackers to discover the database schemas and/or applications.
- Assists with the protection of information from being leaked during an attack on applications.
- Improves the overall security and stability of applications.
- It is recommended by OWASP as an important secure coding practice (OWASP, 2024).

The downside of good error handling as mitigation strategies is referred to below:

- Too vague error messages can make troubleshooting more challenging for developers.
- Additional configuration and management are needed for secure logging systems.
- Insufficiently designed logging can lead to the revelation of sensitive information internally.

- Secure error management might be more complex to develop.

The areas of application of proper error handling as mitigation strategies are mentioned below:

- Web application logic systems
- Database-driven applications
- APIs and cloud services
- E-commerce and banking platforms
- Enterprise and Government systems

6. Conclusion

This coursework is a critical analysis of injection vulnerabilities in Web applications including SQL injection attack and injection-based authentication bypass attacks. The causes, mechanisms, impacts and risks of insecure input handling and vulnerable backend systems were discussed. Theoretical analysis showed how attackers can manipulate vulnerable applications to gain unauthorized access and alter system behaviour.

The hands-on exercise with OWASP Juice Shop effectively demonstrated the impact that malicious payloads can have within authentication systems and revealed security vulnerabilities within web apps. Vulnerability assessment was conducted both manually and automatically by using security testing tools like Postman, SQLmap, OWASP ZAP, and the Developer Tools of the Browser in a safe and secure environment. The results indicated that certain vulnerabilities might not be identified by automated scanners and manual penetration testing might be necessary and in-depth technical analysis may be needed.

The coursework also tested several mitigation techniques such as parameterized queries, input validation and sanitization, Web Application Firewalls, least privilege principles, secure API security practices and proper error handling. The results of the practical WAF implementation showed that the mechanisms of protection at run-time worked well and prevented malicious requests from being accepted while permitting legitimate requests.

In summary, the course material illustrates that it is still one of the biggest security concerns with web applications today that can be easily injected with data. To effectively safeguard web apps against the ongoing cyber threats, organizations need to employ secure coding practices, regular vulnerability assessment, multi-layered defense frameworks, and ongoing security monitoring.

References

A. Bucko, K. V. B. K. a. B. R., 2023. Enhancing JWT authentication and authorization in web applications based on user behavior history. *Computers*, 12(4).

Clarke, J., 2012. *SQL Injection Attacks and Defense*. 2nd edn. Burlington: Syngress. [Online]

Available at: [https://books.google.com.np/books?hl=en&lr=&id=Spm7UgBwzjIC&oi=fnd&pg=PR3&dq=Clarke,+J.+\(2012\)+SQL+Injection+Attacks+and+Defense.+2nd+edn.+Burlington:+Syngress&ots=k25DWMeikH&sig=cpyTGD0NDf5Jdl1PZn0ebE_PeTQ&redir_esc=y#v=onepage&q&f=false](https://books.google.com.np/books?hl=en&lr=&id=Spm7UgBwzjIC&oi=fnd&pg=PR3&dq=Clarke,+J.+(2012)+SQL+Injection+Attacks+and+Defense.+2nd+edn.+Burlington:+Syngress&ots=k25DWMeikH&sig=cpyTGD0NDf5Jdl1PZn0ebE_PeTQ&redir_esc=y#v=onepage&q&f=false)

[Accessed 19 April 2026].

Halfond, W. V. J. a. O. A., 2006. *A classification of SQL-injection attacks and countermeasures*, *Proceedings of the IEEE International Symposium on Secure Software Engineering*. [Online]

Available at: <https://viterbi-web.usc.edu/~halfond/papers/halfond06issse.pdf>

[Accessed 19 April 2026].

Jones, M. B. J. a. S. N., 2015. : *Internet Engineering Task Force*. [Online]

Available at: <https://datatracker.ietf.org/doc/html/rfc7519>

[Accessed 30 April 2026].

Markel, D., 2014. Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*, 2014(239), p. 2.

Michael Jones, J. B. a. N. S., 2015. *JSON Web Token (JWT)*. [Online]

Available at: <https://datatracker.ietf.org/doc/html/rfc7519>

[Accessed 30 April 2026].

MITRE, 2024. *CWE-89: Improper Neutralization of Special Elements used in an SQL Command* ('SQL Injection'). [Online]

Available at: <https://cwe.mitre.org/data/definitions/89.html>

[Accessed 7 May 2026].

Neupane, S., 2025. *Detecting and Mitigating SQL Injection Vulnerabilities in Web Applications*. [Online]

Available at: <https://arxiv.org/pdf/2506.17245>

[Accessed 30 April 2026].

NIST, 2022. *Secure Software Development Framework (SSDF)*. [Online]

Available at: <https://csrc.nist.gov/search?keywords=Secure+Software+Development+Framework+%28SSDF%29&ipp=25&sortBy=relevance&showOnly=publications%2Cprojects%2Cnews%2Cevents%2Cpresentations%2Cglossary%2Ctopics&topicsMatch=ANY&status=Fin>

al%2CDraft

[Accessed 7 May 2026].

Oracle, 2024. *Using Prepared Statements*. [Online]
Available at: <https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html>
[Accessed 7 May 2026].

OWASP, 2021. *OWASP Top 10: A03 Injection*. [Online]
Available at: https://owasp.org/Top10/2021/A03_2021-Injection/index.html
[Accessed 19 April 2026].

OWASP, 2023. *OWASP Juice Shop Project*. [Online]
Available at: <https://owasp.org/www-project-juice-shop/>
[Accessed 30 April 2026].

OWASP, 2024. *Error Handling Cheat Sheet*. [Online]
Available at: https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html
[Accessed 9 May 2026].

OWASP, 2024. *Input Validation Cheat Sheet*. [Online]
Available at: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
[Accessed 9 May 2026].

OWASP, 2025. *SQL Injection Prevention Cheat Sheet*. [Online]
Available at: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
[Accessed 9 May 2026].

PortSwigger, 2023. *SQL Injection*. [Online]
Available at: <https://portswigger.net/web-security/sql-injection>
[Accessed 19 April 2026].

PortSwigger, 2023. *SQL Injection*. [Online]
Available at: <https://portswigger.net/web-security/sql-injection>
[Accessed 21 April 2026].

Postman, 2026. *API Platform for Building and Using APIs*. [Online]
Available at: https://apidog.com/blog/use-postman-free/?utm_source=google_dsa&utm_medium=g&utm_campaign=23049168614&utm_content=185527570709&utm_term=&gad_source=1&gad_campaignid=23049168614&gbraid=0AAAAA-gKXrAjWZ7lVglZFXx9xme6sxK-C&gclid=CjwKCAjwwJzPBhBREiwAJfHRnQNj9V
[Accessed 21 April 2026].

Postman, I., 2024. *Postman API Platform*. [Online]
Available at: <https://www.postman.com/solutions/api-documentation/>
[Accessed 01 May 2026].

Projects, P., 2024. *Documentation*. [Online]
Available at: <https://flask.palletsprojects.com/en/stable/>
[Accessed 7 May 2026].

PTES, 2014. *penetration testing standard*. [Online]
Available at: http://www.pentest-standard.org/index.php/Main_Page
[Accessed 28 April 2026].

Security, O., 2024. *Kali Linux Documentation*. [Online]
Available at: <https://www.kali.org/docs/>
[Accessed 30 April 2026].

Sherekar, B. &., 2022. SQL injection attack: Detection and prevention techniques. *International Journal of Engineering Research and Technology (IJERT)*, 11(4), pp. 812-819.

sqlmap, 2026. *sqlmap project homepage*. [Online]
Available at: <https://sqlmap.org/>
[Accessed 21 April 2026].

Stuttard, D. a. P. M., 2011. *The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws*. 2nd Edition ed. Indianapolis: Wiley.

Weidman, G., 2014. *Penetration Testing: A Hands-On Introduction to Hacking*. 1st Edition ed. San Francisco: No Starch Press.

ZAP, O., 2026. *The world's most widely used web app scanner*. [Online]
Available at: <https://www.zaproxy.org/search/?q=The+world%E2%80%99s+most+widely+used+web+app+scanner>
[Accessed 21 April 2026].